A BLUEPRINT FOR LEAN FORMALIZATION OF THE GENERALIZED STAR-HEIGHT PROGRAM

RYUYA HORA AND THE ZMC GENERALIZED STAR-HEIGHT WORKING GROUP

Abstract. This document specifies a staged Lean formalization for research on the generalized star-height problem. It separates established results, formalization interfaces, computational certificates, and speculative research routes. The first formal milestones are words and languages, generalized regular-expression semantics, deterministic automata, monoid recognition, syntactic congruence, and a trusted certificate boundary. The mathematical program proceeds from published height-one constructions through the first unresolved order-twelve cases A_4 and Dic_3, then to the non-solvable test case A_5. Group or monoid cohomology is included only as an exploratory layer requiring a concrete map from recognition data to cochains. The blueprint also gives theorem signatures, dependency graphs, test obligations, failure modes, and release gates for an AI-assisted workshop.

Contents

Date: 22 July 2026.
2020 *Mathematics Subject Classification.* 68V15, 03B35, 20B35, 20J06.
Key words and phrases. generalized star height, regular languages, syntactic monoids, finite groups, Lean, certificates, A_5.

1. Purpose, scope, and status discipline

1.1. **What this blueprint is.** The goal is not to make a large formal-language library before knowing which theorem will matter. The goal is to build the smallest reliable formal kernel that can support three kinds of research artifact:

(1) a checked upper-bound certificate, usually an explicit generalized regular expression of star-height at most one;
(2) a formalized transport theorem, such as closure under an inverse morphism or a substitution;
(3) a sound lower-bound invariant or game, should one be found.

The formalization is therefore organized around theorem interfaces and verification boundaries rather than around encyclopedic coverage.

1.2. **Status labels.** Every substantial claim in the repository is assigned one of the following labels.

PROVED	proved in Lean or in a complete argument stored in the repository, with a location;
CITED	taken from a primary source whose exact hypotheses have been checked;
COMPUTED	obtained by a deterministic program with a run manifest and independently checkable output;
CONJECTURAL	a precise mathematical statement not proved;
SPECULATIVE	a research vocabulary or route that is not yet a precise conjecture;
REFUTED	false, with an explicit counterexample or source correction;
UNREVIEWED	generated or imported and awaiting audit.

A Lean declaration containing `sorry` is an interface and remains UNREVIEWED. A finite search that fails to find an expression remains COMPUTED; it is not a lower bound.

1.3. **Current mathematical status.** A generalized regular expression permits complement in addition to the ordinary regular operations. The decision problem asks for the minimum nesting depth of Kleene star. No regular language of generalized star-height greater than one is known in the surveyed literature; the decision problem remains open [PST92; Bou17; PZ17]. This is distinct from restricted star height, where complement is unavailable and the decision problem was settled by Hashiguchi [Has88].

The workshop adopts the following north-star conjecture.

Conjecture 1.3.1 (Height-one collapse)**.** *For every finite alphabet A and every regular language $L \subseteq A^*$, the generalized star-height of L is at most one.*

A proof of theorem 1.3.1 would reduce exact height computation to the established height-zero question: a regular language is star-free exactly when its syntactic monoid is aperiodic [Sch65; MP71]. A counterexample would need an explicit regular language and a rigorous argument excluding every height-one expression.

2. Mathematical objects to formalize

2.1. **Words and languages.**

Fix a type A of letters. A word is a finite list, so the free monoid is

$$A^* := \mathrm{List}(A), \qquad 1_{A^*} = \varepsilon,$$

with multiplication given by concatenation. A language is a subset $L \subseteq A^*$. The formalization uses

Listing 1. Core Lean representations.

```
abbrev Word (A : Type u) := List A
abbrev Language (A : Type u) := Set (Word A)
```

This choice makes the empty word and complement universe explicit. In particular, L^c always means $A^* \setminus L$, never complement inside a finite sample of words.

Language concatenation and star are semantic definitions:

$$LK = \{uv \mid u \in L,\ v \in K\}, \qquad L^* = \bigcup_{n \geq 0} L^n, \quad L^0 = \{\varepsilon\}.$$

The first tests must include the empty alphabet, $\varnothing^* = \{\varepsilon\}$, and associativity of concatenation.

2.2. **Generalized regular expressions.**

Definition 2.2.1. For a finite alphabet A, the type $\mathrm{GRegex}(A)$ is inductively generated by

$$0, \quad \varepsilon, \quad a\ (a \in A), \quad r + s, \quad rs, \quad r^c, \quad r^*.$$

Its denotation $[\![r]\!] \subseteq A^*$ is defined compositionally.

The syntax-tree height is

$$\begin{aligned} \mathrm{sh}(0) = \mathrm{sh}(\varepsilon) = \mathrm{sh}(a) &= 0, \\ \mathrm{sh}(r + s) = \mathrm{sh}(rs) &= \max(\mathrm{sh}(r), \mathrm{sh}(s)), \\ \mathrm{sh}(r^c) &= \mathrm{sh}(r), \\ \mathrm{sh}(r^*) &= 1 + \mathrm{sh}(r). \end{aligned}$$

This is not yet the height of a language.

Definition 2.2.2 (Semantic upper bound)**.** For $L \subseteq A^*$ and $n \in \mathbb{N}$, write

$$\mathrm{HasHeightAtMost}(L, n) :\Longleftrightarrow \exists r,\ [\![r]\!] = L \text{ and } \mathrm{sh}(r) \leq n.$$

This existential formulation avoids defining a minimum over an inhabited set before regularity has been established. A later theorem may prove existence and uniqueness of the least n for regular L.

2.3. **Automata.**

A deterministic automaton consists of a state type Q, transition $\delta : Q \times A \to Q$, start state q_0, and accepting set $F \subseteq Q$. The recursive run function must satisfy

$$\delta^*(q, uv) = \delta^*(\delta^*(q, u), v).$$

This lemma is the first nontrivial compile target because it occurs in every proof relating automata, recognition, and contexts. Finiteness should be a separate hypothesis: the core semantics does not need Q finite, while equivalence algorithms do.

2.4. **Monoid recognition.** A finite monoid M recognizes $L \subseteq A^*$ when there is a monoid morphism

$$\varphi : A^* \longrightarrow M$$

and a subset $P \subseteq M$ with $L = \varphi^{-1}(P)$. Surjectivity is not part of recognition. If desired, one may replace M by the image afterward.

Warning 2.4.1. "L is recognized by M" does not mean that the syntactic monoid of L is isomorphic to M. Recognition is existential and is stable under passage to suitable divisors; the syntactic monoid is the minimal recognizing quotient.

The central group-language property is formalized as

$$\mathsf{H}_1(G) :\Longleftrightarrow \text{every language recognized by the finite group } G \text{ has height at most one.}$$

All alphabet, finiteness, morphism, and accepting-set quantifiers must be visible in the Lean definition.

2.5. **Syntactic congruence.** For a language $L \subseteq A^*$, define

$$u \equiv_L v \quad \Longleftrightarrow \quad \forall x, y \in A^*,\ xuy \in L \Longleftrightarrow xvy \in L.$$

The proof order is important:

(1) prove reflexivity, symmetry, and transitivity;
(2) prove compatibility with adding the same prefix and suffix;
(3) prove two-sided compatibility with concatenation;
(4) only then construct the quotient monoid $A^*/\equiv_L$.

Delaying the quotient isolates elementary list algebra from API-sensitive quotient machinery.

3. Source-audited mathematical baseline

3.1. **Height zero.** Schützenberger's theorem gives the algebraic base case: a regular language is star-free if and only if its syntactic monoid is aperiodic [Sch65]. McNaughton and Papert supply the counter-free automata and first-order logical perspective [MP71]. The initial Lean artifact is a theorem interface, not an attempt to formalize all equivalences at once.

3.2. **Pin–Straubing–Thérien.** The principal algebraic source is [PST92]. Its relevant contributions include bounded-height closure under specified quotients, inverse alphabetic morphisms, and injective star-free substitutions; height-one results for languages recognized by commutative groups; extensions to nilpotent groups of class at most two; and a family involving semidirect products by elementary abelian 2-groups. Exact formal statements must be extracted from the paper before use: an abstract or survey paraphrase is not enough for a Lean theorem.

The proof architecture can be separated into three layers:

$$\boxed{\text{explicit counts}} \longrightarrow \boxed{\text{group/monoid decomposition}} \longrightarrow \boxed{\text{closure transport}}.$$

This separation is ideal for formalization because each layer has independent tests.

3.3. **Factor-count certificates.** Bourne and Ruškuc give explicit exact-count and modular-count constructions and apply them to Rees zero-matrix semigroups over abelian groups [BR16]. These expressions are preferable early formalization targets to a purely existential theorem: a concrete expression can be checked by automata equivalence and later imported as a Lean certificate.

3.4. **The order-twelve barrier.** Bourne's thesis records that, in the finite-group ladder under consideration, all groups of order below 12 are covered by preceding results; the remaining order-twelve cases are

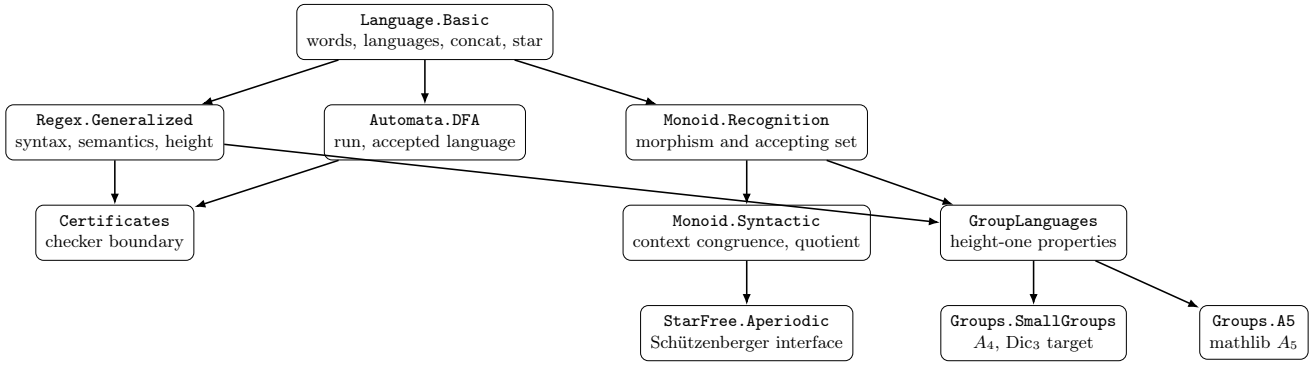
$$A_4 \cong (C_2 \times C_2) \rtimes C_3, \qquad \mathrm{Dic}_3 \cong C_3 \rtimes C_4$$

in the thesis notation [Bou17]. The attempted extension of an $A \rtimes C_2$ argument to $A \rtimes C_3$ fails because the required decomposition of words into consecutive non-overlapping factors is not unique.

This changes the milestone order. A_5 is a strategically important simple non-solvable group, but it is not the first unresolved group-order case. A theorem for A_4, Dic_3, or a precisely specified $A \rtimes C_3$ family is the lower-risk research target.

4. Repository architecture

4.1. Dependency graph.



The graph is intentionally shallow. Cohomology is absent because no concrete theorem yet requires it.

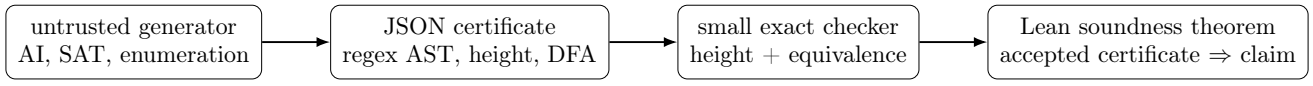
4.2. Module contracts.

Module	Contract
`Language.Basic`	Define words and languages, semantic operations, and elementary membership lemmas. No automata or finiteness assumptions.
`Regex.Generalized`	Define the generalized syntax, denotation, syntactic star height, regularity, star-free, and semantic height upper bounds.
`Automata.DFA`	Define deterministic run and accepted language; prove run over append. Algorithms are placed in a later finite-state file.
`Monoid.Recognition`	Define recognition as inverse image under a list-monoid homomorphism. Keep surjectivity optional.
`Monoid.Syntactic`	Prove contextual equivalence is a two-sided congruence; construct quotient monoid only after this proof.
`StarFree.Aperiodic`	Define aperiodicity and expose a precise interface for Schützenberger's theorem.
`Certificates`	Mirror a finite expression/DFA certificate and prove checker soundness. Generated expressions remain untrusted.
`GroupLanguages.Basic`	State $\mathsf{H}_1(M)$ and $\mathsf{H}_1(G)$ with all quantifiers.
`Groups.SmallGroups`	Give a concrete A_4 model and a vetted canonical model or presentation for Dic$_3$.
`Groups.A5`	Reuse `alternatingGroup (Fin 5)` and existing simplicity/subgroup infrastructure from mathlib [mat26a].

5. Certificate-first verification

5.1. **Why certificates are necessary.** An AI or enumerator can propose a height-one expression much faster than humans can inspect a large Boolean regular expression. The generator should therefore be outside the trusted base. It returns a finite certificate containing the alphabet, abstract syntax tree, claimed height, and target DFA. A small checker:

(1) parses the expression;
(2) computes syntax-tree height;
(3) compiles the expression to a complete DFA, including complement;
(4) minimizes or otherwise checks equivalence with the target DFA;
(5) returns a shortest distinguishing word when equivalence fails.



The bootstrap contains a standard-library-only Python implementation of the first three stages. It compiles generalized expressions by deterministic and ε-NFA constructions rather than by bounded-word testing. Its tests

include a star-free expression for words ending in a and a height-one expression for parity of the number of a's. This program is auditable but not part of Lean's trusted kernel.

5.2. **Lean soundness shape.** The eventual theorem should have the form

Listing 2. Soundness boundary, schematic Lean.

```
theorem checker_sound
  (C : RegexCertificate A Q)
  (h : check C = true) :
  HasHeightAtMost C.target.language C.claimedHeight
```

The difficult theorem is not the last implication; it is proving that the Boolean checker implements denotation, height, and automata equivalence correctly. Reflection can then verify large generated expressions without trusting the generator.

5.3. **Reproducibility metadata.** Every nontrivial computational run records:

commit hash, prompt/input hash, toolchain, resource bounds, output hash, rerun command.

A negative bounded search may be useful as a bug detector or prioritization signal, but its claim is only that no certificate was found within the stated search space.

6. Closure theorems as reusable formal infrastructure

6.1. **Operations.** The Pin–Straubing–Thérien program transports explicit expressions through language operations. Each formal closure theorem needs an exact syntactic constructor or a semantics-preserving transformation. Priority operations are:

(1) left and right quotient by a word;
(2) inverse image under alphabetic morphisms;
(3) injective star-free substitutions under the source's exact hypotheses;
(4) Boolean operations;
(5) products or wreath-product constructions used in the algebraic decomposition.

The formal result should construct a certificate, not merely show existence whenever the paper supplies an explicit construction. For example:

Listing 3. Preferred shape for a transport theorem.

```
def inverseAlphabeticCertificate
  (f : A -> B) (r : GRegex B) : GRegex A := ...

theorem denote_inverseAlphabeticCertificate : ...
theorem height_inverseAlphabeticCertificate :
  (inverseAlphabeticCertificate f r).starHeight <= r.starHeight := ...
```

This split makes semantics and height control independently testable.

6.2. **Do not overstate closure.** A natural-language sentence such as ́bounded height is closed under substitutionís unsafe. Formalization must expose whether the morphism is alphabetic, non-erasing, injective, or star-free; whether the operation is direct or inverse image; and whether the bound is preserved exactly or increased by a constant. The source-audit task precedes the Lean task.

7. Small finite groups before A_5

7.1. A_4 **and** Dic_3**.** The small-group track should encode recognition data for a fixed finite group independently of any global theorem. For A_4, mathlib's alternating-group model is immediate. For Dic_3, the group lead and Lean lead must agree on a canonical model: a semidirect product $C_3 \rtimes C_4$, a permutation subgroup, or a presented finite group with a proved equivalence.

The intended theorem interface is

LISTING 4. Group-language milestone.

```
def HeightOneForGroup (G : Type v) [Group G] : Prop :=
  forall (A : Type u) [Fintype A] [DecidableEq A],
    forall R : Recognition A G,
      HasHeightAtMost R.language 1
```

The proof should expose an algorithm or construction mapping recognition data to an expression certificate whenever possible.

7.2. **Repairing the C_3 obstruction.** The documented failure is not merely "the proof is hard": it is a specific loss of unique factorization. Formalization can help by turning the attempted factorization into a predicate and producing minimal ambiguous words. Replacement mechanisms should be tested separately:

- a canonical parsing automaton with finite synchronization state;
- an unambiguous rational transduction;
- marked factorization forests;
- bounded-overlap inclusion–exclusion;
- a proof that no parser of a specified finite-state normal form can work.

A no-go theorem for a precise normal form is a legitimate result and prevents repeated AI rediscovery of the same failure.

8. The A_5 track

8.1. **Scope.** Mathlib provides A_5 as `alternatingGroup (Fin 5)` and proves simplicity [mat26a]. This removes routine group construction but not the formal-language theorem. Proving $\mathsf{H}_1(A_5)$ would show that height-one methods reach a non-solvable recognizing group; it would not settle theorem 1.3.1.

8.2. **Concrete finite data.** The formal layer may safely compute or import checked tables for:

- the five conjugacy classes: identity, double transpositions, 3-cycles, and two classes of 5-cycles;
- maximal subgroup types A_4, D_5, and S_3;
- the natural action on five points and actions on cosets of selected subgroups;
- restriction of a recognition morphism to preimages of subgroup/coset data.

These tables are research coordinates, not a proof. The central missing operation is a sound gluing theorem turning local certificates into a global certificate.

8.3. **Restriction/gluing matrix.**

Local object	Existing language mechanism	Algebraic data	Required global artifact
cyclic C_2, C_3, C_5	modular counts	characters/cyclic quotients	compatible expression certificates
Klein four	abelian and class-two formulas	elementary 2-group action	overlap control inside A_4
S_3, D_5	semidirect by C_2 families	normal cyclic subgroup	restriction/transfer formula
A_4	unresolved C_3 extension case	$V_4 \rtimes C_3$	repaired parser or obstruction
whole A_5	none known	simple group, coset actions	a gluing theorem or lower-bound invariant

9. Cohomology: formalize only after a bridge exists

9.1. **What cohomology could organize.** For an extension

$$1 \longrightarrow N \longrightarrow G \longrightarrow Q \longrightarrow 1,$$

low-dimensional group cohomology records invariants, crossed homomorphisms, and extension classes; the Lyndon–Hochschild–Serre spectral sequence organizes their interaction [Bro82; Wei94]. A useful language theorem might have the schematic form

$$\mathsf{H}_1(N) + \mathsf{H}_1(Q) + \text{controlled cocycle} \Longrightarrow \mathsf{H}_1(G).$$

This formula is only a research prompt until every term after the plus signs is defined in language-theoretic data.

9.2. **Mandatory bridge data.** Before importing a cohomology library, require:

(1) a specific coefficient module V and explicit G-action;
(2) a map from a word, automaton path, transition count, or accepting fiber to a cochain;
(3) a proof of the cocycle identity;
(4) a theorem describing at least one generalized-expression constructor under this map;
(5) an example where the class is nontrivial and an example where it vanishes.

Without these, úse the LHS spectral sequenceís vocabulary rather than a mathematical mechanism.

9.3. **Candidate coefficient objects.** Candidates include permutation modules on cosets or states, functions on accepting fibers, the augmentation ideal of a group algebra, modules generated by transition-count functions, and bimodules attached to syntactic monoids. Each candidate should be attacked first: complement or concatenation may destroy the proposed invariant. A counterexample at this stage is valuable.

10. A lower-bound formalization track

Upper-bound constructions dominate the known finite-group program. A disproof of height-one collapse requires a language invariant satisfied by all height-one expressions. The formal interface should mirror an induction on syntax:

(1) base cases for $0, \varepsilon$, and letters;
(2) closure under union and complement;
(3) closure or controlled composition under concatenation;
(4) one-level star applied only to star-free operands.

The last two clauses carry the burden. A proposed invariant should be tested against known height-one group languages before candidate search begins.

Possible—but currently speculative—forms include an expression game, profinite identities, a finite-index congruence, a topological invariant of the relevant Boolean algebra, or a cohomological obstruction with a proved constructor calculus. Formalization is useful because it forces the claimed closure theorem to expose every assumption.

11. Testing and review plan

11.1. **Unit tests.** The first test suite includes:

- empty and singleton alphabets;
- $\varnothing$, $\{\varepsilon\}$, universal language, and singleton letters;
- $\varnothing^* = \{\varepsilon\}$ and $\{\varepsilon\}^* = \{\varepsilon\}$;
- run-over-append for automata;
- recognition by the trivial monoid;
- contextual equivalence on a language with a two-state minimal automaton;
- a star-free certificate using complement of the empty language as A^*;
- a height-one parity certificate;
- a deliberately false DFA certificate returning a shortest witness.

11.2. **Property and finite exhaustive tests.** For small finite alphabets and state sets, executable tests may compare:

- expression semantics and compiled automata up to a bounded word length, as a bug detector only;
- direct DFA product operations and language Boolean operations;
- syntactic-equivalence classes computed from a minimal DFA;
- group morphism evaluation and automaton transition monoids.

Bounded testing supplements, but never replaces, the exact checker or Lean proof.

11.3. **Adversarial checklist.** Every theorem-level review asks:

(1) Is star height generalized or restricted?
(2) Is a syntax height being mistaken for the language minimum?
(3) Is complement relative to the correct A^*?

(4) Is the empty word handled?
(5) Is recognition being confused with syntactic-monoid equality?
(6) Are inverse morphism, quotient, subgroup, and divisor directions correct?
(7) Does a factorization claim prove uniqueness or unambiguity?
(8) Is local subgroup information being glued without a theorem?
(9) Is a finite search being used as a lower bound?
(10) Does every cited theorem match the exact hypotheses?

12. Milestones and acceptance gates

12.1. **Conference milestone.** A realistic two-day goal is:

> Compile the core definitions; verify the Python certificate examples; reconstruct one published height-one construction; and isolate the C_3 factorization failure as a precise predicate with an explicit ambiguity.

A new A_4 or A_5 theorem is an ambitious bonus, not a scheduling assumption.

12.2. **Twelve-week sequence.**

Weeks	Deliverable	Gate
1–2	compiling foundation and certificate tests	clean CI, source/semantics review
3–4	one Bourne–Ruškuc expression encoded	exact DFA equivalence certificate
5–6	$A \rtimes C_2$ dependency graph and C_3 ambiguity suite	independent reconstruction
7–8	parsing repair or no-go theorem	prover/breaker agreement
9–10	A_5 subgroup/coset matrix and one concrete bridge candidate	no unsupported gluing claim
11–12	preprint-grade lemma/theorem or obstruction report	external domain review and clean build

12.3. **Announcement gate.** A result advertised as Lean-formalized must build under the pinned toolchain with no undeclared `sorry` or `axiom`. A mathematical resolution additionally requires a complete dependency graph, source audit, independent reconstruction, adversarial review, deterministic reproduction of computations, and external expert checking. The layered review described in OpenAI's public unit-distance report is a useful model: automated generation and grading were followed by internal human examination, external specialists, and human-edited exposition [Ope26b]. The lesson is the independence of verification layers, not that an AI grade is sufficient.

13. AI-assisted proof engineering

13.1. **Prompt architecture.** Durable project instructions belong in small repository files such as `AGENTS.md`; tests and linters enforce them [Ope26a]. Individual tasks must freeze a claim, name nonexamples, list required artifacts, and give a verification command. Anthropic's engineering guidance favors simple composable workflows before elaborate autonomous architectures and emphasizes executable feedback and context management [Ant24; Ant26].

The public Jacobian prompt is useful for its process design: multiple incompatible routes, delayed convergence, theorem-strength missing lemmas, adversarial passes, and a final synthesis phase [Lou26]. For a resource-constrained workshop, use fewer agents but preserve mechanism-level diversity and a protected verification budget.

13.2. **Lean version policy.** The repository pins Lean and mathlib to version 4.32.0, the stable Lean release dated 13 July 2026 in the official release notes [Lea26]. The Lake file uses a fixed mathlib revision and the bootstrap invokes the documented cache command `lake exe cache get` [mat26b]. Toolchain upgrades occur on a separate branch with a complete build and API-change log.

14. Risk register

Risk	Symptom	Mitigation / acceptance test
Wrong problem	restricted-star-height theorem cited as generalized	every claim carries a height convention; source audit
Syntax/semantics confusion	one expression's height presented as a lower bound	use theorem 2.2.2; lower bounds require universal exclusion

Risk	Symptom	Mitigation / acceptance test
Recognition confusion	theorem assumes syntactic monoid equals a chosen group	formalize recognition data and image reduction explicitly
Quotient fragility	time lost on quotient APIs before congruence lemmas	prove list/context compatibility first
Parser illusion	C_3 argument silently assumes unique factors	finite ambiguity suite and explicit parser theorem
Cohomology by analogy	spectral sequence named without a coefficient object	block imports until mandatory bridge data exists
Search overclaim	no small expression found	label as bounded computation with manifest
Formalization theater	polished PDF but Lean contains holes	proof-hole linter and clean CI gate
Agent convergence	all agents repeat the favored route	preserve early independence and approach registry
Context exhaustion	long transcript replaces durable state	checkpoints and task directories
Social pressure	premature announcement or invisible verification labor	independent referee, contribution log, release checklist

Appendix A. Initial proof-obligation map

ID	Obligation	Acceptance
L-WORD-001	core language operations and membership lemmas	file compiles
L-REGEX-001	generalized syntax, semantics, height	file compiles; edge cases reviewed
L-DFA-001	automaton run and append theorem	file compiles; smoke theorem
L-REC-001	monoid recognition	source notation comparison
L-SYN-001	syntactic setoid and two-sided congruence	all congruence lemmas compile
L-SYN-002	quotient monoid	multiplication well-defined
L-SF-001	Schützenberger interface	exact source theorem mapped
L-CERT-001	verified certificate checker	acceptance implies language equality and height bound
L-GRP-001	group-language height-one property	all quantifiers approved
L-A5-001	A_5 model and library facts	pinned mathlib build
M-C3-FAIL-001	explicit factorization ambiguity	word/factors independently checked
N-LOWER-001	height-one invariant	closure under every constructor

Appendix B. Definition acceptance sheet

Before merging a definition, the domain lead and Lean lead answer:

(1) What mathematical object does the type represent?
(2) Which finiteness and decidability assumptions are intrinsic, and which are algorithmic?
(3) What is the ambient complement universe?
(4) What are the empty and degenerate cases?
(5) Which equality is definitional and which requires a theorem?
(6) What is the smallest theorem that exercises the definition?
(7) Does the definition force a later theorem to use choice or quotients unnecessarily?
(8) Is there an executable representation and a semantic representation, and how are they related?

Acknowledgement. The document uses the package, theorem-environment, bibliography, and macro organization of the supplied Ryuya Hora `FirstVersion.tex` template, adapted only for portable fonts and workshop-specific packages. The project should credit separately the mathematical, counterexample, formalization, source-audit, and integration contributions recorded in `CONTRIBUTIONS.md`.

References

[Ant24] Anthropic. *Building Effective Agents*. 2024. URL: https://www.anthropic.com/engineering/building-effective-agents.

[Ant26] Anthropic. *Best Practices for Claude Code*. Accessed 22 July 2026. 2026. URL: https://code.claude.com/docs/en/best-practices.

[Bou17] Thomas Bourne. "Counting Subwords and Other Results Related to the Generalised Star-Height Problem for Regular Languages". PhD thesis. University of St Andrews, 2017. URL: https://hdl.handle.net/10023/12024.

[BR16] Thomas Bourne and Nik Ruškuc. "On the Star-Height of Subword Counting Languages and Their Relationship to Rees Zero-Matrix Semigroups". *Theoretical Computer Science* 653 (2016), pp. 87–96. DOI: 10.1016/j.tcs.2016.09.024. URL: https://hdl.handle.net/10023/11811.

[Bro82] Kenneth S. Brown. *Cohomology of Groups*. Vol. 87. Graduate Texts in Mathematics. Springer, 1982. DOI: 10.1007/978-1-4684-9327-6.

[Has88] Kosaburo Hashiguchi. "Algorithms for Determining Relative Star Height and Star Height". *Information and Computation* 78.2 (1988), pp. 124–169. DOI: 10.1016/0890-5401(88)90021-2.

[Lea26] Lean FRO. *Lean Release Notes*. Accessed 22 July 2026. 2026. URL: https://lean-lang.org/doc/reference/latest/releases/.

[Lou26] Aaron Lou. *Jacobian Conjecture Prompt*. Publicly shared multi-agent prompt, accessed 22 July 2026. 2026. URL: https://aaronlou.com/jacobian_counterexample_prompt.pdf.

[mat26a] mathlib Community. *Mathlib.GroupTheory.SpecificGroups.Alternating and .Simple*. Accessed 22 July 2026. 2026. URL: https://leanprover-community.github.io/mathlib4_docs/Mathlib/GroupTheory/SpecificGroups/Alternating.html.

[mat26b] mathlib Community. *Using mathlib4 as a Dependency*. Accessed 22 July 2026. 2026. URL: https://github.com/leanprover-community/mathlib4/wiki/Using-mathlib4-as-a-dependency.

[MP71] Robert McNaughton and Seymour A. Papert. *Counter-Free Automata*. MIT Press, 1971.

[Ope26a] OpenAI. *Custom Instructions with AGENTS.md*. Accessed 22 July 2026. 2026. URL: https://learn.chatgpt.com/docs/agent-configuration/agents-md.

[Ope26b] OpenAI. *Planar Point Sets with Many Unit Distances*. Includes the original problem prompt and a statement on automated generation and layered review. 2026. URL: https://cdn.openai.com/pdf/74c24085-19b0-4534-9c90-465b8e29ad73/unit-distance-proof.pdf.

[PST92] Jean-Éric Pin, Howard Straubing, and Denis Thérien. "Some Results on the Generalized Star-Height Problem". *Information and Computation* 101.2 (1992), pp. 219–250. DOI: 10.1016/0890-5401(92)90063-L. URL: https://hal.science/hal-00019978.

[PZ17] Thomas Place and Marc Zeitoun. *Generic Results for Concatenation Hierarchies*. 2017. arXiv: 1710.04313 [cs.FL]. URL: https://arxiv.org/abs/1710.04313.

[Sch65] Marcel-Paul Sch"utzenberger. "On Finite Monoids Having Only Trivial Subgroups". *Information and Control* 8.2 (1965), pp. 190–194. DOI: 10.1016/S0019-9958(65)90108-7.

[Wei94] Charles A. Weibel. *An Introduction to Homological Algebra*. Cambridge University Press, 1994. DOI: 10.1017/CBO9781139644136.